Graphics and Multimedia

Graphics

If one talks about standard components that we can be used in UI, this is good but it is not enough when we want to develop a game or an app that requires graphic contents. Android SDK provides a set of API for drawing custom 2D and 3Dgraphics. When we write an app that requires graphics, we should consider how intensive the graphic usage is. In other words, there could be an app that uses quite static graphics without complex effects and there could be other app that uses intensive graphical effects like games. According to this usage, there are different techniques we can adopt:

• **Canvas and Drawable:** In this case, we can extend the existing UI widgets so that we can customize their behavior or we can create custom 2D graphics using the standard method provided by the Canvas class.

• Hardware acceleration: We can use hardware acceleration when drawing with the Canvas API. This is possible from Android 3.0. Hardware acceleration is enabled by default if your Target API level is >=14, but can also be explicitly enabled. If your application uses only standard views and Drawables, turning it on globally should not cause any adverse drawing effects. However, because hardware acceleration is not supported for all of the 2D drawing operations, turning it on might affect some of your custom views or drawing calls. Problems usually manifest themselves as invisible elements, exceptions, or wrongly rendered pixels. To remedy this, Android gives you the option to enable or disable hardware acceleration at multiple levels.

Controlling Hardware Acceleration

You can control hardware acceleration at the following levels:

- Application
- Activity
- Window
- View

Application level

In your Android manifest file, add the following attribute to the <application> tag to enable hardware acceleration for your entire application:

<applicationandroid:hardwareAccelerated="true" ...>

Activity level

If your application does not behave properly with hardware acceleration turned on globally, you can control it for individual activities as well. To enable or disable hardware acceleration at the activity level, you can use the android:hardwareAccelerated attribute for the <activity> element. The following example enables hardware acceleration for the entire application but disables it for one activity:

```
<applicationandroid:hardwareAccelerated="true">
        <activity ... />
        <activity android:hardwareAccelerated="false"/>
 </application>
```

Determining if a View is Hardware Accelerated

It is sometimes useful for an application to know whether it is currently hardware accelerated, especially for things such as custom views. This is particularly useful if your application does a lot of custom drawing and not all operations are properly supported by the new rendering pipeline. There are two different ways to check whether the application is hardware accelerated:

- View.isHardwareAccelerated() returns true if the View is attached to a hardware accelerated window.
- Canvas.isHardwareAccelerated() returns true if the Canvas is hardware accelerated

If you must do this check in your drawing code, use Canvas.isHardwareAccelerated() instead ofView.isHardwareAccelerated() when possible. When a view is attached to a hardware

accelerated window, it can still be drawn using a non- hardware accelerated Canvas. This happens, for instance, when drawing a view into a bitmap for caching purposes.

• OpenGL:

Android supports OpenGL natively using NDK. This technique is very useful when we have an app that usesintensively graphic contents (i.e games).

Android includes support for high performance 2D and 3D graphics with the Open Graphics Library (OpenGL®), specifically, the OpenGL ES API. OpenGL is a cross- platform graphics API that specifies a standard software interface for 3D graphics processing hardware. OpenGL ES is a flavor of the OpenGL specification intended for embedded devices. Android supports several versions of the OpenGL ES API:

- OpenGL ES 1.0 and 1.1 - This API specification is supported by Android 1.0 and higher.
- OpenGL ES 2.0 - This API specification is supported by Android 2.2 (API level 8) and higher.
- OpenGL ES 3.0 - This API specification is supported by Android 4.3 (API level 18) and higher.
- OpenGL ES 3.1 - This API specification is supported by Android 5.0 (API level 21) and higher.

The easiest way to use 2D graphics is extending the Viewclass and overriding the onDraw method. We can use this techniquewhen we do not need a graphics intensive app. In this case, we can use the Canvas class to create 2D graphics. This class provides a set of method starting with draw thatcan be used to draw different shapes like:

• lines

• circle

• rectangle

• oval

• picture

• arc

For example let us suppose we want do draw a rectangle. We create a custom view and then we override onDraw method. Here we draw the rectangle:

```
public class CustomView extends View {

    private Rect rectangle;
    private Paint paint;

    public CustomView(Context context) {
        super(context);
        int x = 50;
        int y = 50;
        int sideLength = 200;

        // create a rectangle that we'll draw later
        rectangle = new Rect(x, y, sideLength, sideLength);

        // create the Paint and set its color
        paint = new Paint();
        paint.setColor(Color.GRAY);
    }

    @Override
    protected void onDraw(Canvas canvas) {
        canvas.drawColor(Color.BLUE);
        canvas.drawRect(rectangle, paint);
    }
}


public class MainActivity extends Activity {
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(new CustomView(this));
    }

}
```

Animation

Animation is the process of creating the illusion of motion and shape change by means of the rapid display of a sequence of static images that minimally differ from each other. The illusion—as in motion pictures in general is thought to rely on the phi phenomenon. Animators are artists who specialize in the creation of animation. Animation can be recorded on either analogue media, such as a flip book, motion picture film, video tape, or on digital media, including formats such as animated GIF, Flash animation or digital video. To display animation, a digital camera, computer, or projector are used along with new technologies that are produced.

Animation creation methods include the traditional animation creation method and those involving stop motion animation of two and three-dimensional objects, such as paper cutouts, puppets and clay figures. Images are displayed in a rapid succession, usually 24, 25, 30, or 60 frames per second. Animations can add subtle visual cues that notify users about what's going on in your app and improve their mental model of your app's interface. Animations are especially useful when the screen changes state, such as when content loads or new actions become available. Animations can also add a polished look to your app, which gives your app a higher quality feel.

Crossfading Two Views

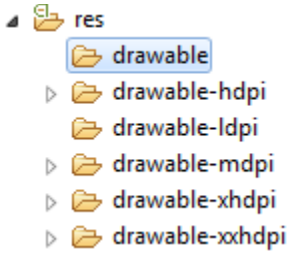
Crossfade animations (also known as dissolve) gradually fade out one UI component while simultaneously fading in another. This animation is useful for situations where you want to switch content or views in your app. Crossfades are very subtle and short but offer a fluid transition from one screen to the next. When you don't use them, however, transitions often feel abrupt or hurried.

Using ViewPager for Screen Slides

Screen slides are transitions between one entire screen to another and are common with UIs like setup wizards or slideshows. The images slides with a ViewPager provided by the support library.ViewPagers can animate screen slides automatically. Here's what a screen slide looks like that transitions from one screen of content to the next.

Drawable

In Android, a Drawable is a graphical object that can be shown on the screen. From API point of view all the Drawable objects derive from Drawable class. They have an important role in Android programming and we can use XML to create them. They differ from standard widgets because they are not interactive, meaning that they do not react to user touch. Images, colors, shapes, objects that change their aspect according to their state, object that can be animated are all drawable objects. In Android under res directory, there is a sub-dir reserved for Drawable, it is called res/drawable.



Under the drawable dir we can add binary files like images or XML files. We can create several directories according to the screen density we want to support. These directories have a name like drawable-<….>. This is very useful when we use images; in this case, we have to create several image versions: for example, we can create an image for the high dpi screen or another one for medium dpi screen. Once we have our file under drawable directory, we can reference it, in our class, using

R.drawable.file_name

While it is very easy add a binary file to one of these directory, it is a matter of copy and paste, if we want to use a XML file we have to create it.

There are several types of drawable:

• Bitmap

• Nine-patch

• State list

• Level list

• Transition drawable

• Inset drawable

• Clip drawable

• Scale drawable

• Shape drawable

Shape drawable

This is a generic shape. Using XML we have to create a file with shape element as root. This element as an attribute called android:shape where we define the type of shape like rectangle, oval, line and ring. We can customize the shape using child elements like: For example, let us suppose we want to create an oval with solid background color. We create a XML file called for example oval.xml:

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
android:shape="oval">
        <solid android:color="#FF0000" />
        <size android:height="100dp" android:width="120dp" />
 </shape>
```

Multi touch and gestures

Multi-touch gesture happens when more then one finger touches the screen at the same time. Android allows us to detect these gestures. Android system generates the following touch events whenever multiple fingers touches the screen at the same time.

Sr.No	Event & description
1	**ACTION_DOWN** For the first pointer that touches the screen. This starts the gesture.
2	**ACTION_POINTER_DOWN** For extra pointers that enter the screen beyond the first.
3	**ACTION_MOVE** A change has happened during a press gesture.
4	**ACTION_POINTER_UP** Sent when a non-primary pointer goes up.
5	**ACTION_UP** Sent when the last pointer leaves the screen.

So in order to detect any of the above mention event, you need to override onTouchEvent() method and check these events manually. Its syntax is given below –

```
Public Boolean onTouchEvent(MotionEventev){

Final intactionPeformed=ev.getAction();

switch(actionPeformed){

 case MotionEvent.ACTION_DOWN:{ break; }

case MotionEvent.ACTION_MOVE:{ break; }

return true; }
```

In these cases, you can perform any calculation you like. For example zooming , shrinking e.t.c. In order to get the co-ordinates of the X and Y axis, you can call getX() and getY() method. Its syntax is given below:

final float x =ev.getX();

final float y =ev.getY();

Apart from these methods, there are other methods provided by this MotionEvent class for better dealing with multitouch. These methods are listed below:

Sr.No	Method & description
1	**getAction()** This method returns the kind of action being performed
2	**getPressure()** This method returns the current pressure of this event for the first index
3	**getRawX()** This method returns the original raw X coordinate of this event
4	**getRawY()** This method returns the original raw Y coordinate of this event
5	**getSize()** This method returns the size for the first pointer index
6	**getSource()** This method gets the source of the event
7	**getXPrecision()** This method return the precision of the X coordinates being reported
8	**getYPrecision()** This method return the precision of the Y coordinates being reported

Multimedia

The Android SDK provides a set of APIs to handle multimedia files, such as audio, video and images. Moreover, the SDK provides other API sets that help developer's to implement interesting graphics effects, like animations and so on.

The modern smart phones and tablets have an increasing storage capacity so that we can store music files, video files, images. etc. Not only the storage capacity is important, but also the high

definition camera makes it possible to take impressive photos. In this context, the Multimedia API plays an important role.

Multimedia API

Android supports a wide list of audio, video and image formats. You can give a look here to have an idea; just to name a few formats supported:

Audio

• MP3

• MIDI

• Vorbis (es: mkv)

Video

• H.263

• MPEG-4 SP

Images

• JPEG

• GIF

• PNG

All the classes provided by the Android SDK that we can use to add multimedia capabilities to our apps are under the android.media package.

In this package, the heart class is called **MediaPlayer**. This class has several methods that we can use to play audio and video file stored in our device or streamed from a remote server. This class implements a state machine with well-defined states and we have to know them before playing a file. Simplifying the state diagram, as shown in the official documentation, we can define these macro-states:

• Idle state: When we create a new instance of the MediaPlayer class.

• Initialization state: This state is triggered when we use setDataSource to set the information source that MediaPlayer has to use.

• Prepared state: In this state, the preparation work is completed. We can enter in this state calling prepare method or prepareAsync. In the first case after the method returns the state moves to Prepared. In the async way, we have to implement a listener to be notified when the system is ready and the state moves to Prepared. We have to keep in mind that when calling the prepare method, the entire app could hang before the method returns because the method can take a long time before it completes its work, especially when data is streamed from a remote server. We should avoid calling this method in the main thread because it might cause a ANR (Application Not Responding) problem. Once the MediaPlayer is in prepared state we can play our file, pause it or stop it.

• Completed state: To end of the stream is reached.

We can play a file in several ways:

```
// Raw audio file as resource
 MediaPlayermp = MediaPlayer.create(this, R.raw.audio_file);
// Local file
MediaPlayer mp1 = MediaPlayer.create(this, Uri.parse("file:///...."));
 // Remote file
MediaPlayer mp2 = MediaPlayer.create(this, Uri.parse("http://website.com"));
```

or we can use setDataSource in this way:

```
 MediaPlayer mp3 = new MediaPlayer();
 mp3.setDataSource("http://www.website.com");
```

Once we have created our MediaPlayer we can “prepare” it:

```
mp3.prepare();
```

and finally we can play it:

mp3.start();

Using Android Camera

If we want to add to our apps the capability to take photos using the integrated smart phone camera, then the best way is to use an Intent. For example, let us suppose we want to start the camera as soon as we press a button and show the result in our app. In the onCreate method of our Activity, we have to setup a listener of the Button and when clicked to fire the intent:

```
Button b = (Button) findViewById(R.id.btn1);
 b.setOnClickListener(new View.OnClickListener() {
 @Override
public void onClick(View v) {
// Here we fire the intent to start the camera
Intent i = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
startActivityForResult(i, 100);

} });
```

In the onActivityResult method, we retrieve the picture taken and show the result:

```
@Override
protected void onActivityResult(intrequestCode, intresultCode, Intent data) {
// This is called when we finish taking the photo
```

```
Bitmap bmp = (Bitmap) data.getExtras().get("data");

iv.setImageBitmap(bmp);

}
```